

x402

Hakron

Hi, I'm Sara Jane 🖐️

/ My passion

→ (Learning) & Teaching people the possibilities of
blockchain technology



@SARAJANE2004

LinkedIn:



Integration Specialist @ Algorand Foundation

Contents session 1



What is Algorand?

- / Facts & Stats
- / The technology
- / Integrations

X402 on Algorand

- / What is the x402 Standard?
- / Walkthrough
- / Live demo

Agentic Commerce

- / Why does it matter?
- / Key use-cases & Market size
- / How does crypto fit in?

Wrapping up

- / Relevant ARCs
- / Ideas on what to build with x402



What is Algorand?

Origins & key differentiators

--- Algorand is

**an energy efficient, quantum secure,
Layer-1 blockchain with instant finality,
consistently high throughput, and low
fees. It's been operating at 100% uptime
for seven years.**

Algorand today: by the numbers



>10,000

Transactions per second

< 3

Second block time

0.001

Algo transaction fees

0

Seconds to reach transaction finality

0

Forks

0

Downtime

>58 million

Blocks

>3.4 billion

Transactions

~770K

Monthly active addresses

*data recorded February 2026

Algorand technology is different



Instantly Final

With no forking and instant settlement finality, users face not even one millisecond of uncertainty as to whether their transaction will achieve on-chain finality.

Scalable

Algorand's pure proof-of-stake consensus mechanism enables global scale. It handles over 10,000 transactions per second, at a sub-three-second block time.

Reliable

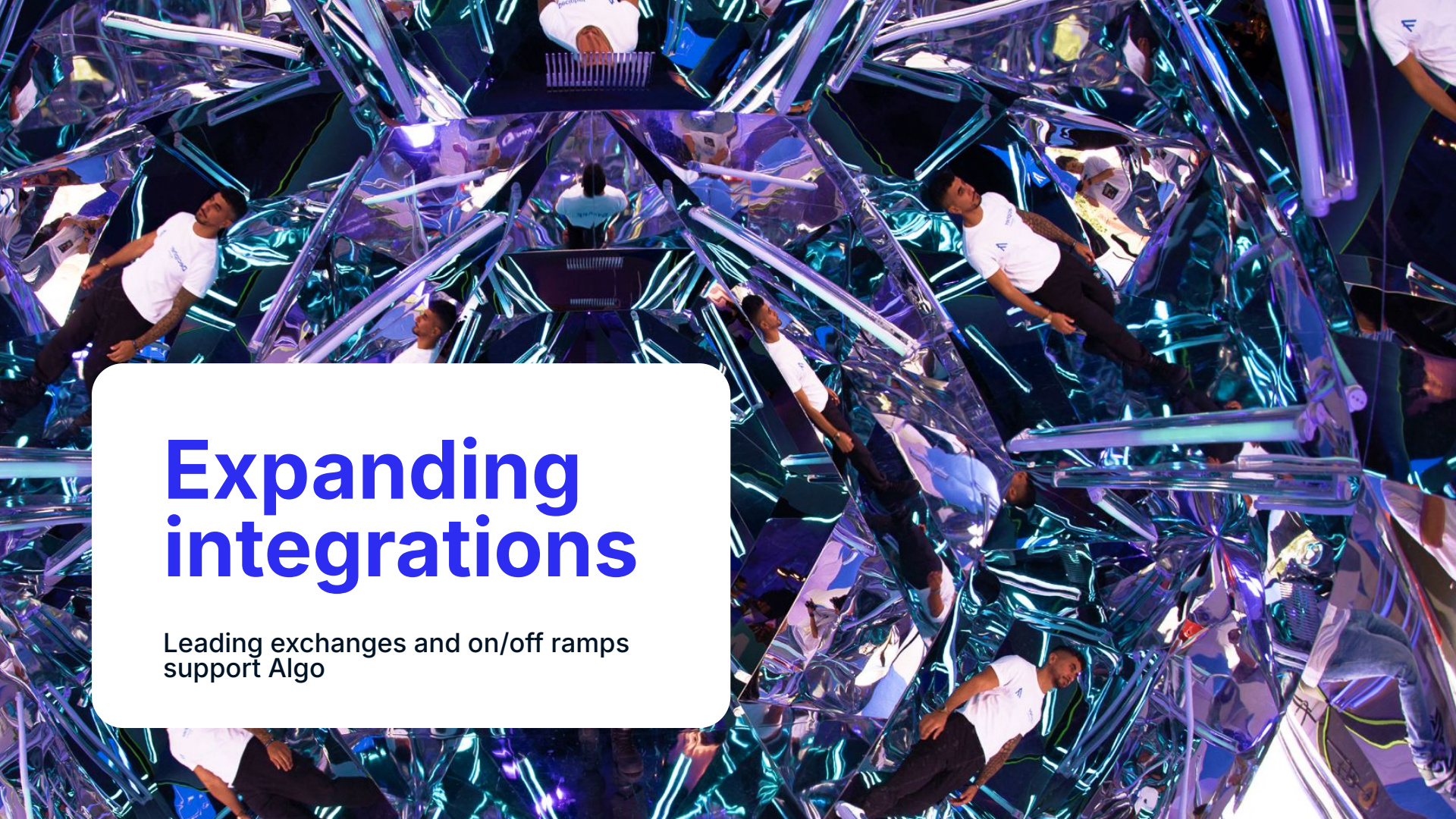
Its peer-to-peer network topography, decentralized security model, and flawless performance history makes Algorand one of the most reliable and resilient blockchain platforms in the world.

Permissionless

Algorand validator nodes can be run on a consumer-grade laptop. Enterprise-grade hardware is not required. Powering the entire Algorand network requires around 140 kW, the equivalent of accelerating a Tesla.

Quantum Secure

Algorand generates a "State Proof" - utilizing lattice-based Falcon signatures every 256 blocks (about 10-15 minutes) - that secures the history of the ledger against quantum threats. Developers can also utilize Falcon to execute quantum-secure transactions on Algorand mainnet.



Expanding integrations

Leading exchanges and on/off ramps support Algo

Algo is supported by the top exchanges



More than 80 exchanges support Algo

Coinbase ✓

Binance ✓

Bybit ✓

OKX ✓

Upbit ✓

Kraken ✓

Kucoin ✓

Gate.io ✓

BitGP ✓

BitMart ✓

HTX ✓

Bitfinex ✓

MEXC ✓

Bitstamp ✓

Crypto.com ✓

LBank ✓

BingX ✓

Coinstore ✓

LCX ✓

Bitget ✓

USDCa is supported by the top exchanges



Exchanges supporting USDC on Algorand

Binance



Coinbase



Kraken



MEXC



Kucoin



HTX



Crypto.com



Bittrue



Coinstore



Top on/off ramps support Algo & USDC



Alchemy Pay



TransFi



Onramp.money



Transak



MoonPay



CoinGate

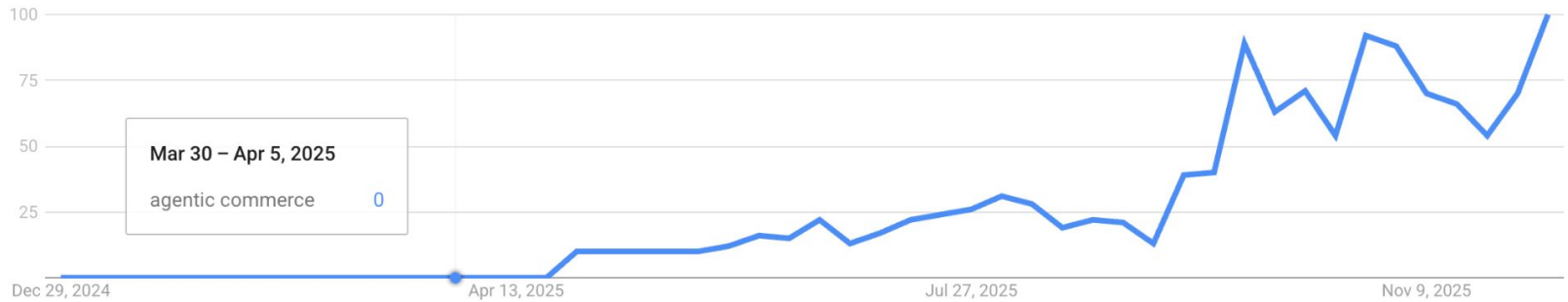
A person wearing a blue hoodie and a lanyard is seated, holding a smartphone in their left hand and writing with a pen in their right hand on a lined notebook. The background is slightly blurred, showing other people in a similar setting.

Agentic Commerce

When blockchain makes sense



Interest over time ?



Why does it matter?

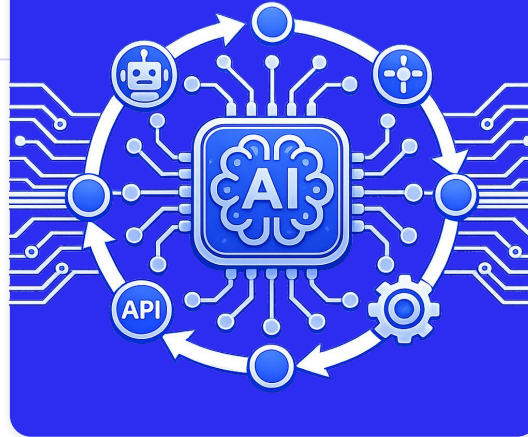


Human-initiated Commerce

- / UI-driven
- / Slow, expensive coordination

Agentic Commerce

- / Machine-to-machine interaction
- / API-driven
- / Continuous optimization



Second-Order Effects

- / Long-tail markets become viable
- / Marginal decision cost $\rightarrow \sim 0$
- / Capital allocates itself in real time

Expected market size



Global Market Size Projections (by 2030)

**\$3 trillion
to \$5 trillion**

in global orchestrated revenue is forecast by McKinsey & Company.

**\$10 to
\$12 trillion**

in annual online sales, or roughly a quarter of all e-commerce, is what Kearney projects AI shopping agents could drive

**\$1.7 trillion
by 2030**

with a 67% CAGR from 2025, is Citi Ventures' estimate for the total addressable market (TAM) of C2B agentic commerce

**50% of
its revenue**

within the next five years is what Walmart believes will come from AI agents

U.S. Market Size Projections (by 2030)

**Up to
\$1 trillion**

in orchestrated revenue is what McKinsey estimates the U.S. B2C retail market alone could see

**\$300 to
\$500 billion**

or roughly 15% to 25% of overall e-commerce, is what Bain & Company estimates for the U.S. agentic commerce market

**\$190 billion
to \$385 billion**

in U.S. e-commerce spending by 2030 is what Morgan Stanley estimates AI-powered 'agentic shoppers' could represent



How does crypto fit in?

Payment and trust layer
for Agentic Commerce

/ **Payment rail:** instant, cheap, and feasible for small micro-transactions.

/ **Programmable:** smart contracts allow for instant multi-party settlement, escrowed intents, etc.

/ **Transparency and trust:** agent identities, reputation, and transaction history are recorded on-chain and can be verified and audited.

/ **Traceability:** every transaction and history of interactions is traceable and immutable all the time.

Emerging standards & protocols

Emerging protocols and standards on agentic identity and trust, payment rails, how Agents communicate with external resources, among themselves and with end user interfaces

MCP (Model Context Protocol)



A2A (Agent to Agent Communications)

X402 (Web 2.0 like Web 3.0 payments)

AP2 (Agents Payments Protocol)

ERC 8004 (Trustless Agents Spec)

AGUI (Agentic GUI Protocol)

A2UI (Agents To UI declarative spec)

UCP (Universal Commerce Protocol)



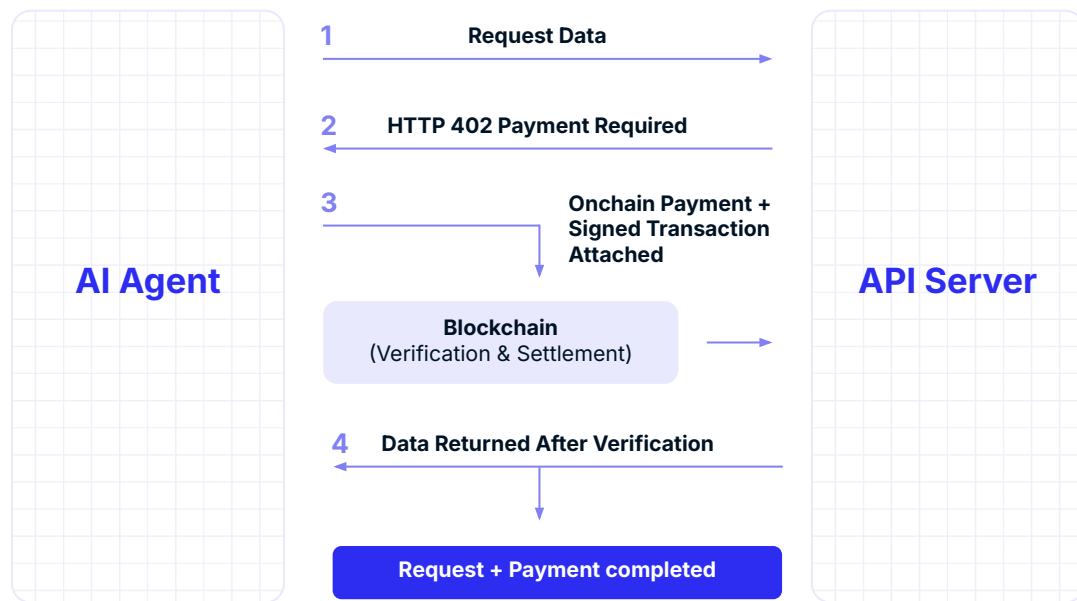
Introducing x402 payments on Algorand!

What does this mean?

X402 - Internet-native payments standard



Introduction



/ New standard, proposed by Coinbase, to enable autonomous AI agent payments over blockchain rails.

/ Based on HTTP error code 402 "payment required", it embeds transaction capability into the core structure of the Internet.

/ Enables micropayments at scale.

/ Key use cases:
/ Pay-per-API access
/ On-demand compute access
/ Access to premium data/content

Walkthrough

X402 - Internet-native payments standard





Live demo time!

Lets build the walkthrough

Right before, let's look at the code!



Client side:

```
import { config } from 'dotenv';
import { x402Client, wrapFetchWithPayment, x402HTTPClient } from '@x402/fetch';
import { toClientAvmSigner, ExactAvmScheme, ALGORAND_TESTNET_CAIP2 } from '@x402/avm';
import {
  ed25519SigningKeyFromWrappedSecret,
  type WrappedEd25519Seed,
} from '@algorandfoundation/algokit-utils/crypto';
import { seedFromMnemonic } from '@algorandfoundation/algokit-utils/algo25';

config();

const avmMnemonic = process.env.AVM_MNEMONIC as string;
const url = 'https://x402.goplausible.xyz/examples/weather';
```

Right before, let's look at the code!



Client side:

```
async function main(): Promise<void> {  
  const secretKey = await getSecretKeyFromMnemonic(avmMnemonic);  
  // Create the Algorand signer used to authorize payments.  
  const avmSigner = toClientAvmSigner(secretKey);  
  
  // Initialize the x402 client.  
  const client = new x402Client();  
  
  // Register with network  
  client.register(ALGORAND_TESTNET_CAIP2, new ExactAvmScheme(avmSigner));  
  
  console.info(`AVM signer: ${avmSigner.address}`);  
}
```


Right before, let's look at the code!



Client side:

```
// Wrap fetch so 402 responses trigger payment handling automatically.
const fetchWithPayment = wrapFetchWithPayment(fetch, client);

const response = await fetchWithPayment(url, { method: 'GET' });

if (response.ok) {
  const paymentResponse = new x402HTTPClient(client).getPaymentSettleResponse(name =>
    response.headers.get(name),
  );
  console.log('\nPayment response:', JSON.stringify(paymentResponse, null, 2));
} else {
  console.log(`\nNo payment settled (response status: ${response.status})`);
}
}
```

Right before, let's look at the code!



Client side:

```
// Build the base64-encoded signing key that x402-avm expects.
// The format is the 32-byte Ed25519 seed concatenated with the 32-byte public key.
async function getSecretKeyFromMnemonic(avmMnemonic: string): Promise<string> {
  const seed = seedFromMnemonic(avmMnemonic);
  const seedCopy = new Uint8Array(seed);
  const wrappedSeed: WrappedEd25519Seed = {
    unwrapEd25519Seed: async () => seed,
    wrapEd25519Seed: async () => {},
  };
  const wrappedSecret = await ed25519SigningKeyFromWrappedSecret(wrappedSeed);

  return Buffer.concat([Buffer.from(seedCopy), Buffer.from(wrappedSecret.ed25519Pubkey)]).
    'base64',
  );
}

main().catch(error => {
  console.error(error?.response?.data?.error ?? error);
  process.exit(1);
});
```

Right before, let's look at the code!



Server side:

```
import { config } from 'dotenv';
import { x402Client, wrapFetchWithPayment, x402HTTPClient } from '@x402/fetch';
import { toClientAvmSigner, ExactAvmScheme, ALGORAND_TESTNET_CAIP2 } from '@x402/avm';
import {
  ed25519SigningKeyFromWrappedSecret,
  type WrappedEd25519Seed,
} from '@algorandfoundation/algokit-utils/crypto';
import { seedFromMnemonic } from '@algorandfoundation/algokit-utils/algo25';

config();

const avmMnemonic = process.env.AVM_MNEMONIC as string;
const url = 'https://x402.goplausible.xyz/examples/weather';
```

Right before, let's look at the code!



Server side:

```
// Initialize the Resource Server  
const facilitatorClient = new HTTPFacilitatorClient({ url: facilitatorUrl });  
const server = new x402ResourceServer(facilitatorClient);  
  
// Register Network Schemes  
const avmServerScheme = new ExactAvmScheme();  
server.register(ALGORAND_TESTNET_CAIP2, avmServerScheme);  
  
const app = new Hono();
```

Right before, let's look at the code!



Server side:

```
// Protected Route Middleware
app.use(
  paymentMiddleware(
    {
      'GET /weather': {
        accepts: [
          {
            scheme: 'exact',
            price: '$0.005',
            network: ALGORAND_TESTNET_CAIP2,
            payTo: avmAddress,
            extra: { asset: USDC_TESTNET_ASA_ID },
          },
        ],
        description: 'Weather data access',
      },
    },
    server,
  ),
);
```

Right before, let's look at the code!



Server side:

```
// Resource Handler
app.get('/weather', c => {
  return c.json({
    report: {
      weather: 'sunny',
      temperature: 70,
      timestamp: new Date().toISOString(),
    },
  });
});

serve({ fetch: app.fetch, port: 4021 }, () => {
  console.log(`x402 Resource Server listening at http://localhost:4021`);
});
```




Let's start coding!

Time to build

But there's more ...



- / **Discovery.** How can agents find other agents with particular skills?
- / **Trust.** In a world full of agents, who can you trust?
- / **Security.** Do agents control their own on-chain spending accounts? How do we prevent them from going rogue?
- / **X-chain collaboration.** How will these protocols, specifically X402, work in a X-chain world?
- / **User Experience.** While the focus has been on the plumbing, a lot still needs to be done to ensure a seamless user experience.

Discovery - Bazaar



Algorand Bazaar is a registry and discovery interface for A2A Agents, X402/AP2 Merchants, Resources and facilitators all in one place, available via API, ABI, MCP and A2A.

/ Agents (</discovery/agents>)

- Onchain registrar of agents with API and ABI accessibility
- Agents discovery and query by bio, URL, skills and capabilities
- [Agents verifiable identity, skills and capabilities](#)

/ Merchants (</discovery/merchants>)

- Merchants (agentic and non agentic) discovery
- Merchant resources and payment methods discovery and query

/ Resources (</discovery/resources>)

- Fixed pricing resources
- Dynamic pricing resources
- Periodically pricing resources
- Conditional pricing resources

/ Payment Methods (/discovery/payment_methods)

- Assets and tokens
- Fee delegation
- Transaction Group blueprints (onchain capabilities)

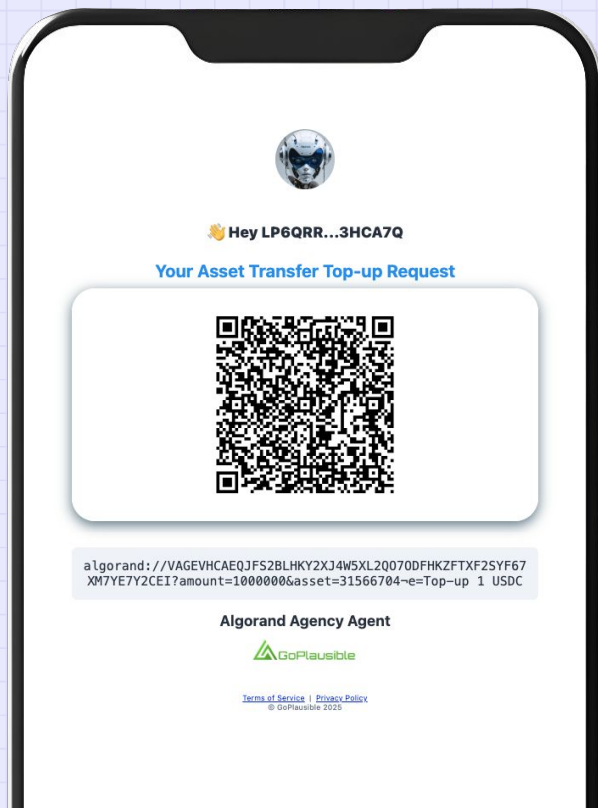
/ Facilitators (</discovery/facilitators>):

- Verify (off-chain and on-chain)
- Settle (on-chain)
- List supported chains and methods
- Availability via API/ABI/MCP/A2A

Security - ARC-90 + Instant Top-ups



Algorand Request for Comment



/ Agent dedicated wallets to be always empty (no risk of funds misuse on hallucination incidents)

/ Usable in both user present and user not present scenarios

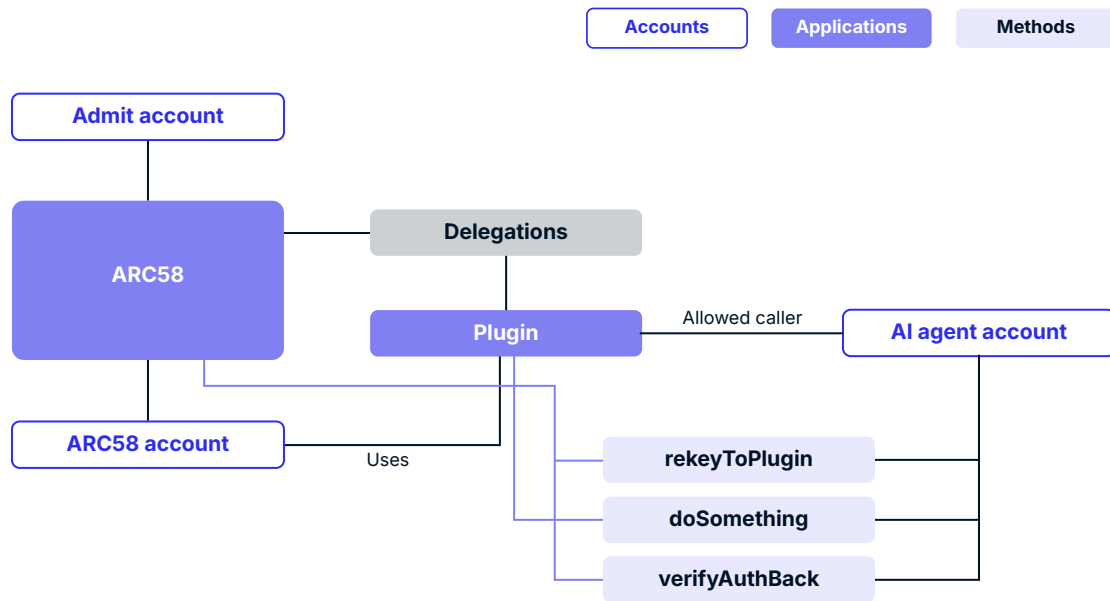
/ Agents will pop top-up QR-Codes, exactly by the required amounts and users scan and topup with one transaction

/ Easiest and least complex for DX and rapid implementations

Security - ARC-58 - account abstraction



Algorand Request for Comment



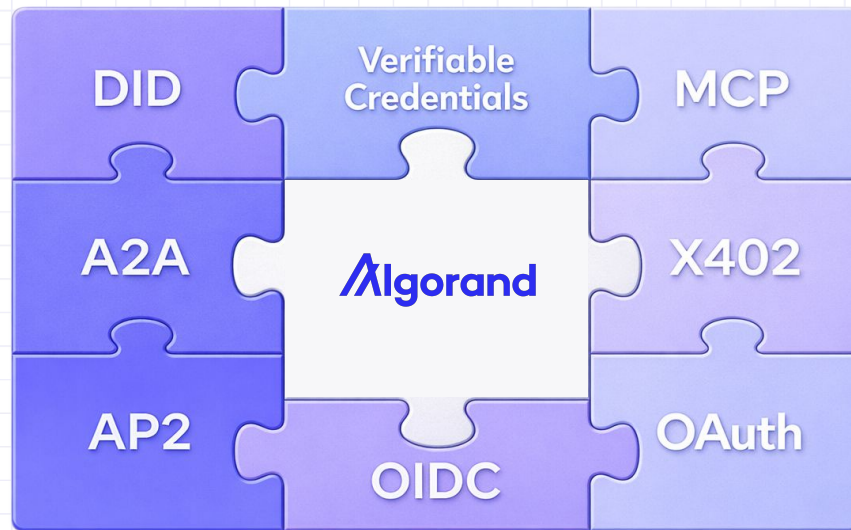
/ Proposes to tackle the security issues associated with giving an agent unbound access to a blockchain-based spending account.

/ Plug-in system constraints an agent to pre-approved transactions in line with agent scope.

/ Flash rekeying combined two unique Algorand features (rekeying and atomic groups) to give an agent access to funds only for the duration of the approved transaction.



Putting it all together



Ideas on what to build



Top 6 use cases of Agentic Commerce



Shopping agents

Pay-per-use agents with stablecoin wallets



Bot-to-Bot payments

Autonomous transacting



Subscriptions

Autonomous payment renewals



Micropayments

Real-time, low-cost transactions



Rewards

Crypto payouts and incentives



Tokenized checkout

Agent-powered single-click purchasing

Speed-first agents with stablecoin wallets

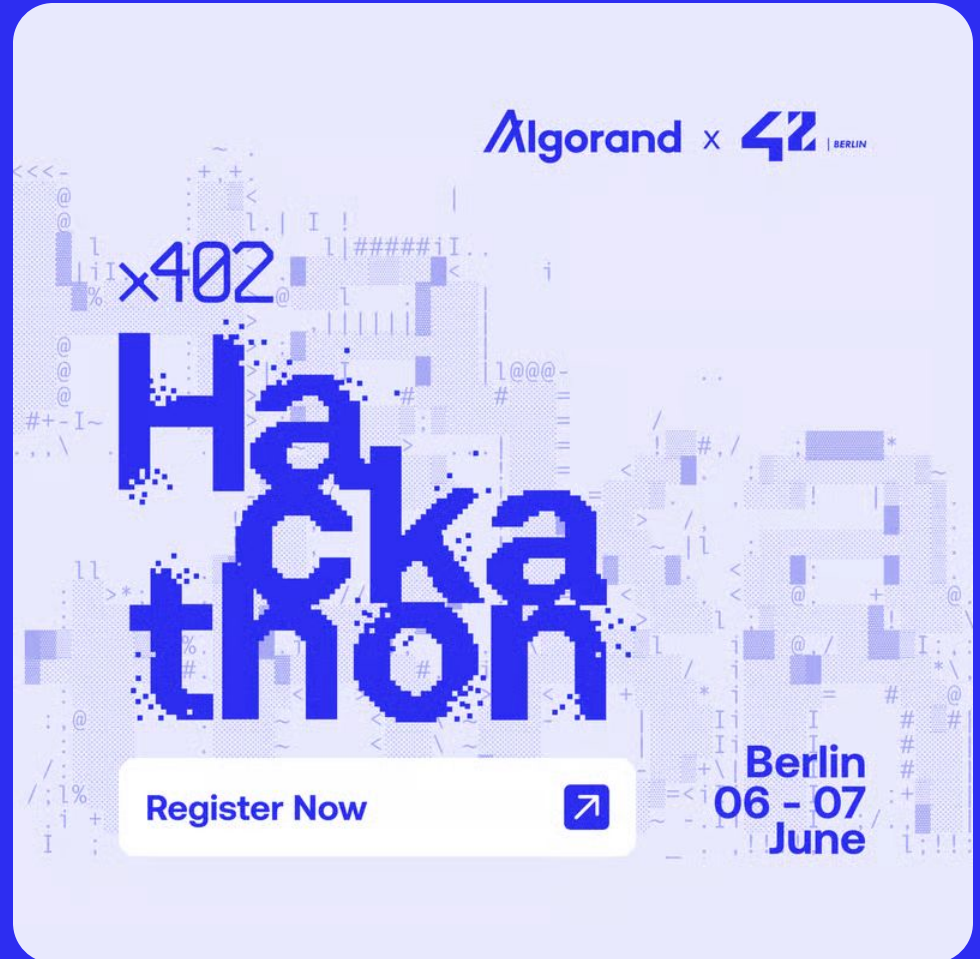


Algorand Builders Berlin

Agentic Commerce x402 Hackathon

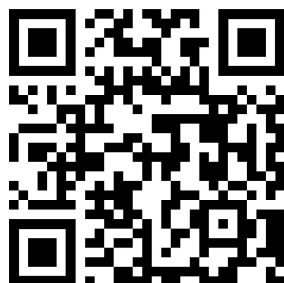
\$20,000+

TOTAL PRIZE POOL



x402
Ha
cka
thon

Thank you



Scan to Register